

## Lab 2: AST Printer

---

Continue developing the language you designed in Lab 1, using Lox and the textbook's implementation as references. Define your language's expression grammar, write its abstract syntax tree (AST) classes, and print hard-coded ASTs. Explain your design choices and differences from Lox in your report. This lab covers AST representation and printing only; parsing and execution are not required.

Python is the preferred implementation language, but you may choose another language if you prefer. Reuse components such as the `Token` class from Lab 1, and include everything needed to run your code.

### Tasks

You will submit both your **code** and a **report** for this lab. Partial credit will be awarded for all tasks.

Report quality and submission correctness together account for at most **10% of the total score**, included within the task weights below.

- **Syntactic grammar design (20%):**

Define your language's expression grammar using the formal notation introduced in class. Include literals, unary expressions, binary expressions, and grouping, using Lox's expression grammar as a reference. Explain the supported operators and literal types, and include the grammar in your report.

- **AST implementation (40%):**

Implement nodes in your abstract syntax tree. Each node must store the fields needed to represent that expression, such as operands, operators, or literal values.

- **AST printer (40%):**

Develop an AST printer that supports every expression class in your language. Use parentheses ( `)` to show the tree's hierarchy, as shown in the lecture slides. Create a test program that constructs and prints hard-coded ASTs using your AST classes.

For example, using Lox-style classes:

```
expression = Binary(  
    Unary(Token(TokenType.MINUS, "-", None, 1), Literal(123)),  
    Token(TokenType.STAR, "*", None, 1),  
    Grouping(Literal(45.67)),  
)
```

The string representation of this tree should be `(* (- 123) (group 45.67))`. Adapt the operators and literal values to your language's grammar as needed.

### Testing and Report

Write at least **three hard-coded AST examples** that collectively cover every expression class, every supported literal type and operator, and nested expressions in your language. Add more examples as

needed to cover edge cases in your AST representation and printer. Verify that your AST classes correctly represent the expressions and that the printer preserves each tree's hierarchy.

Place your report under `report/` in your repository. Your report must include:

- Expression grammar, and design choices relative to Lox.
- Dependency/setup instructions and commands to reproduce the printer tests. If you implement an AST generator for extra credit, also include commands to run it and regenerate the expression classes.
- For each test case: its purpose, the hard-coded AST, expected output, actual printer output, and whether the result matches expectations. Output may be included as text or readable screenshots.
- Any known limitations or failing tests.

Commit the test inputs and any test scripts under `test/lab2/` so that your results can be reproduced.

## Optional Extra Credit

You may apply for extra credit if you believe you put extra effort into a feature beyond the basic AST implementation and printer requirements. Examples include metaprogramming for generating the node classes, an additional tree visualization format, an alternative printer using reverse Polish notation, or AST conversion to and from JSON.

In your report under `report/`, identify the feature, explain the additional effort and its value, and provide test cases and output demonstrating it. Extra credit is subject to instructor evaluation and is capped at **10% of the lab's base grade (10 percentage points)** across all extra-credit features.

## Submission Instructions

Submit a **link to the exact commit in your GitHub repository** on Blackboard, using the format `https://github.com/<owner>/<repo>/commit/<commit-hash>`.

The submitted commit must include:

- All implementation files in `src/`, including the AST classes, printer, and reused Lab 1 components. If you implement an AST generator for extra credit, include both the generator and its generated source files.
- All test inputs and test scripts for this lab in `test/lab2/`.
- The Lab 2 report in `report/`, including the expression grammar, setup/run instructions, test results, and any extra-credit request.

Use the following repository structure (the report filename below is an example):

```
repository/
├── src/           # Implementation files
├── test/
│   └── lab2/     # Lab 2 test inputs
└── report/
    └── lab2.md   # Lab 2 report
```

Ensure that the instructor can access the repository and reproduce your results from the submitted commit.

## Late Penalty

You should submit your lab on time. Being late for one lab could affect your time to complete subsequent labs. All labs are due at 11:59 pm (Eastern) of the specified due date, and there is **a 10% penalty each day for up to 50%**. After that, you get zero.

## AI Policy

Using AI help for the labs is ALLOWED but discouraged. But you must quote the code from AI help.

Quote it like this:

```
// --- start AI code ---
```

```
// --- end AI code ---
```

Using AI generated code without quoting is considered academic misconduct and will result in 0 for the lab. Mistakes in the code from AI help will result in doubled point deductions according to the rubrics.

Reuse of code from previous labs is ALLOWED and encouraged.